Risk Assessment of Insurence Policies using Machine Learning Techniques

Lukáš Trstenský, Bartosz Kochanski and Maia Ghionea

https://github.com/SAKULAK/ITU-MachineLearning-FinalProject-Couriers

Contents

1 Introduction

This paper aims to explore methods to model the claims risk of insurance policies using various machine learning methods as well as discuss their implementation. We will explore and clean the data, then define the claims risk based on the data available in the dataset.

1.1 Machine Learning Methods

In this paper we will use custom implementation of a Feed-Forward Neural Network Regressor and custom implementation of a Decision Tree Regressor using only Python numerical libraries, e.g. Python. Furthermore, we will use an ensemble method using a combination of **TBD**

2 Data

The dataset used in the scopes of this project is the French Motor Third-Part Liability (`freMTPL2freq`) dataset from the `CASdatasets` (Dutang et al., 2024, p. 71) R package. It contains numerical and categorical information on the insurance claims of drivers for their cars spanning multiple regions, details about the cars themselves, such as age, gas type or brand, the drivers, and even the regions where the cars are driven, which more or less impact the risk of a claim being made.

2.1 Data Format

For each unique policy in our dataset, there is a number of claims made within the policy, its duration, then subsequent information on the driver and car. For a clearer overview of the set see Table 1 for the names and corresponding descriptions of the data columns.

Table 1: Data columns and descriptions.

Name	**Description**
IDpol	Unique policy ID.
ClaimNb	Number of claims made in policy.
Exposure	Duration of policy in years.
Area	Density category of driver's residence from "A" for rural area to "F" for urban centre.
VehPower	Power category of car.
VehAge	Age of car.
DrivAge	Age of driver.
BonusMalus	Bonus/malus, between 50 and 350: $<$ 100 means bonus, $>$ 100 means malus.
VehBrand	The car brand divided in the following groups: A- Renaut Nissan and Citroen, B- Volkswagen, Audi, Skoda and Seat, C- Opel, General Motors and Ford, D- Fiat, E- Mercedes Chryslerand BMW, F- Japanese (except Nissan) and Korean, G- other.
VehGas	Type of gas that the vehicle uses.
Density	Number of inhabitants per km^2 in the city the driver of the car lives in.
Region	The policy region in France (based on the 1970-2015 classification).

2.2 Data Cleaning

A conservative approach was taken in the scope of data cleaning, by minimally modify the data. The material did not present any obvious signs of subverting our assumptions, except for the following:

1. Exposure values should lie in the range between 0 and 1; since the data was collected over the span of one year, we assume that any value above 1 is a mistake in the collection and therefore any value over this range will be reduced to 1, the maximum value permitted.
2. Any data points where $VehAge > 30$ are excluded. This is the cut-off mark at which vehicles become classified as historic cars and will in turn be treated differently by car insurance companies.

Despite the rest of the data not presenting any faults, the information left is narrowed down to solely the figures utilized by the models. Namely, the following columns have been dropped: (**EDIT THESE IF CHANGES**)

a. `IDpol` - not a feature

b. `Area` - categorical counterpart of `Density`

c. `VehBrand` - Vehicle brand alone does not indicate risk of policy since a singular brand has many models of vehicle, some of which might attract more risky drivers, however we believe that the risk evens out along the whole lineup of cars from a brand, well in this case group of brands.

d. `Region` - Another indicator of Density.

2.3 Visualization

Figure 1: This is an interesting visualization of the data.

3 Features

So after dropping these we are left with our feature columns: `VehPower`, `VehAge`, `DrivAge`, `BonusMalus`, `VehGas`, `Density`
and the columns on which we base our `Risk` calculations: `ClaimNb` and `Exposure`.

3.1 Claims Risk

How do we calculate it, why (idk)

3.2 PCA & Clustering

Figure 2: PCA with Risk as color — Clustering on PCA data.

4 M1 - Decision Tree

Simple description of DT, probably also mention the way prediction values are assigned here, because I think it's too short to make a section of it, but if you think otherwise do it that way

4.1 Structure

4.2 Finding Splits

4.2.1 Recursive

4.2.2 Priority Queue

4.3 Stopping Parameters

Why, what problems they solve

Maybe an enumerate of stopping params with simple description of how they work, if that fits, if not use subsubsection for each stopping parameter

a. **Max Depth**
This is the description?

b. **Minimum Samples to Split** -

c. **Minimum Samples at Leaf** -

d. **Maximum number of Leaves** -

e. **Minimum Impurity Decrease** -

5 M2 - Feed-Forward Neural Network

A Feed-Forward Neural Network (*FFNN*) is our second custom implemented regression method. In this section we will explain the basics of how a FFNN works along with how we implemented these methods.

5.1 The Structure

A FFNN is a collection of nodes organized in layers. First, the input layer. The input layer number of nodes is equal to the number of features in the dataset. Second, the hidden layers. There can be an arbitrary number of hidden layers and of nodes per hidden layer. Generally, the more hidden layers or nodes per layer the more complex the Neural Network becomes, which makes it more prone to overfitting and more computationally heavy to train and run predictions on. Lastly, the output layer. The number of nodes in the output layer depends on the task, but for regression there is just one node, which give the value predicted value for the data points.

Figure 3: Structure of a FFNN

5.2 The Flow

A FFNN at its core follows a very simple flow, see Figure 4

Figure 4: Flowchart of a FFNN

5.3 Initialization

We initialize an array of weights (W_{ij}) and an array of biases (b_j) for each pair of subsequent layers. The shape of W_ij is the number of nodes in the previous layer (i) by the number of nodes in the current layer (j) which is also the shape of the bias array.

To fill the arrays with initial values we use the He Initialization (He et al., 2015) by drawing from a zero-mean normal distribution with standard deviation of $\sqrt{2/n_i}$ which is considered to be among the best initializations for shallow ReLU based FFNNs.

5.4 Forward Pass

During the Forward Pass, the data, as the name suggests propagates forward through the network. At each layer, except the input layer, the activations of the previous layer (a_i) are weighed by the weights and have the bias added to them:

$$z_j = a_i W_{ij} + b_j$$

Then they are passed through the activation function to become the activation of the current layer, which are then passed to the next layer.

5.4.1 Activation Functions

There are many activation functions that one might choose depending on the task. Usually you use two different activation functions, one for the hidden layers and a different one for the output layer. For hidden layers we use the widely used activation function, the Rectified Linear Unit (*ReLU*) function defined as $ReLU(x) = max(0, x)$. For the output layer of regression FFNNs a simple linear function is used $f(x) = x$

5.5 Back propagation

5.6 Updating Parameters

5.7 Non-Essential Features

5.7.1 Early Stopping

5.7.2 Mini-Batch Gradient Descent

5.7.3 Weight Decay

6 Correctness Testing

To validate our implementation of the Decision Tree and the Feed-Forward Neural Network we performed correctness testing.

6.1 Comparing Against a Dummy Regressor

6.2 Simple Function Test

6.2.1 Continuous Variables

6.2.2 Categorical Variables

7 M3 - Ensemble

Idk let's see what we do

8 Results

Probably some table of results of each method and how and why we evaluated like we did.

9 Limitations

Probably could have somehow cleaned the data better to get better separation of Risk values and would like claim amounts.

10 Conclusion

Hopefully something good. Summarize the results of each method, include running times say which one worked best as a combination of time and performance

10.1 Improvments

everything

References

[Dutang et al., 2024] Dutang, C., Charpentier, A., Gallic, E., and Siharath, J. (2024). CASdatasets: Insurance datasets. `https://cas.uqam.ca/pub/web/CASdatasets-manual.pdf`. R package version 1.2-0.

[He et al., 2015] He, K., Zhang, X., Ren, S., and Sun, J. (2015). Delving deep into rectifiers: Surpassing human-level performance on imagenet classification.